

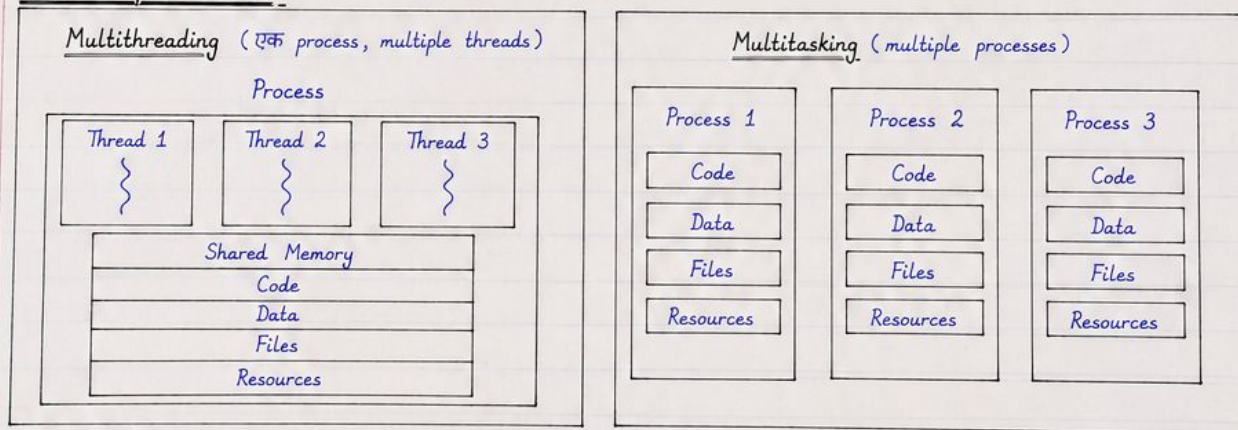
1. Multithreading (मल्टीथ्रेडिंग) :

- Multithreading एक programming concept है जिसमें एक ही process के अंदर multiple threads execute होती हैं।
- Threads एक-दूसरे के साथ memory और resources share करती हैं।
- यह performance improve करता है और application को responsive बनाता है।

Difference Between Multithreading and Multitasking :

Sr. No.	Multithreading	Multitasking
1. Definition	एक ही process के अंदर multiple threads को execute करना।	Multiple processes को एक साथ execute करना।
2. Unit of Execution	Thread	Process
3. Memory Usage	कम memory use होती है क्योंकि threads same memory space share करती हैं।	अधिक memory use होती है क्योंकि हर process का अपना अलग memory space होता है।
4. Context Switching	कम समय लगता है क्योंकि threads के बीच context switch हल्का होता है।	अधिक समय लगता है क्योंकि processes के बीच context switch भारी होता है।
5. Communication	Threads के बीच communication आसान होता है क्योंकि वे same memory share करती हैं।	Processes के बीच communication मुश्किल होता है (IPC की आवश्यकता होती है)।
6. Resource Sharing	Code, Data, Files आदि resources share होते हैं।	Resources share नहीं होते, हर process के अपने resources होते हैं।
7. Creation Overhead	Threads बनाना आसान और कम overhead वाला होता है।	Processes बनाना महंगा और अधिक overhead वाला होता है।
8. Example	Web browser में एक thread UI handle करता है और दूसरा thread data download करता है।	Music player चलाना और साथ में Word document पर काम करना।
9. Dependency	यदि एक thread crash हो जाए तो पूरा process प्रभावित हो सकता है।	यदि एक process crash हो जाए तो अन्य processes प्रभावित नहीं होते।
10. Best For	CPU-bound tasks और performance improvement के लिए उपयुक्त।	I/O-bound tasks और multiple applications एक साथ चलाने के लिए उपयुक्त।

Visual Representation :



Key Points :

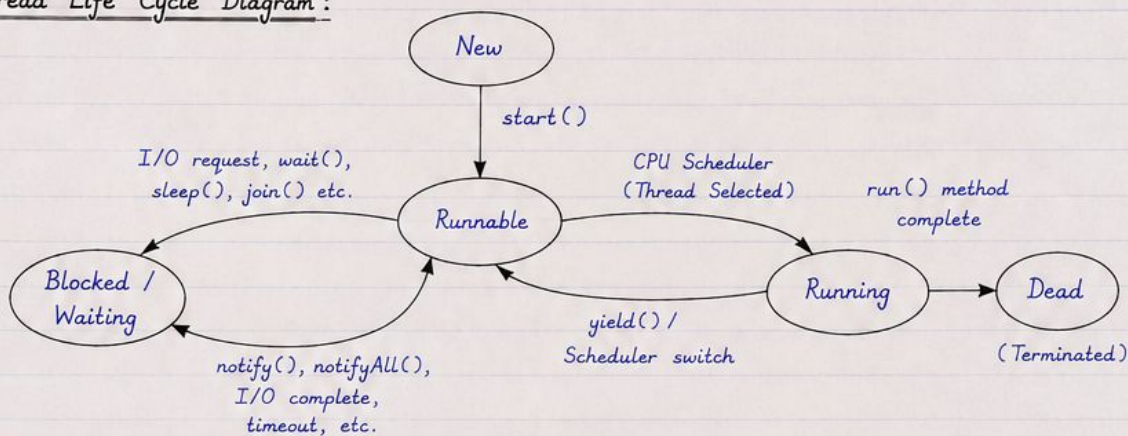
- ✓ Multithreading में एक ही process के अंदर multiple threads parallel रूप से काम करती हैं।
- ✓ Multitasking में OS multiple processes को time slice देकर execution manage करता है।
- ✓ Multithreading performance और responsiveness बढ़ाता है।
- ✓ Multitasking system utilization और productivity बढ़ाता है।

1. Multithreading (मल्टीथ्रैडिंग) :

(A) Thread Life Cycle (थ्रेड जीवन चक्र) :

- जब एक thread create किया जाता है, तब वह New state में होता है।
- start() method call होने पर वह Runnable state में चला जाता है।
- CPU मिलने पर वह Running state में execute होता है।
- किसी कारण से वह Waiting/Blocked state में जा सकता है।
- execution पूरा होने पर thread Dead (Terminated) state में चला जाता है।

Thread Life Cycle Diagram :



Thread Life Cycle States :

State	Description	Enter State	Exit State
1. New	Thread create तो हो गया है लेकिन start() नहीं हुआ है।	new Thread()	Runnable (start() के बाद)
2. Runnable	Thread execute होने के लिए ready होता है और CPU मिलने का wait करता है।	start()	Running / Blocked / Dead
3. Running	Thread को CPU मिल जाता है और वह run() method execute करता है।	Scheduler द्वारा select होने पर	Runnable / Blocked / Dead
4. Blocked / Waiting	Thread किसी resource (I/O, lock, sleep, wait आदि) के लिए wait करता है।	sleep(), wait(), join(), I/O wait	Runnable (resource मिलने पर)
5. Dead (Terminated)	Thread का execution पूरा हो जाता है या terminate हो जाता है।	run() complete या stop()	कोई नहीं (अंतिम अवस्था)

State Transitions (संक्रमण) को समझें :

- New → Runnable : start() method call होने पर.
- Runnable → Running : CPU Scheduler thread को select करता है.
- Running → Runnable : time slice पूरा होने पर या yield() call पर.
- Running → Blocked/Waiting : sleep(), wait(), join(), I/O wait आदि.
- Blocked/Waiting → Runnable : notify(), notifyAll(), timeout या I/O complete होने पर.
- Running → Dead : run() method complete होने पर.

Important Points :

- ✓ एक time पर एक thread को ही CPU मिलता है।
- ✓ Scheduler threads के बीच CPU time allocate करता है।
- ✓ wait() और sleep() दोनों thread को waiting state में डालते हैं, पर wait() में lock release होता है जबकि sleep() में नहीं।
- ✓ Dead state में जाने के बाद thread फिर से start नहीं हो सकता।

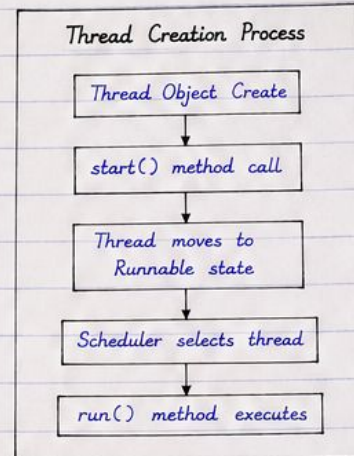
Example (सरल उदाहरण) :

```
Thread t = new MyThread(); // New state
t.start(); // Runnable state
// CPU मिलने पर
// MyThread का run() execute होगा → Running state
// run() complete होने पर → Dead state
```


1. Multithreading (मल्टीथ्रैडिंग):

(A) Creating Threads (थ्रेड बनाना):

- Java में किसी भी thread को create करने के लिए Thread class या Runnable interface का use किया जाता है।
- Thread को start() method द्वारा start किया जाता है। start() method के बाद ही run() method execute होता है।
- एक program में main thread के अलावा हम multiple user defined threads बना सकते हैं।
- Thread बनाना दो तरीकों से किया जा सकता है:
 1. Thread class को extend करके
 2. Runnable interface को implement करके



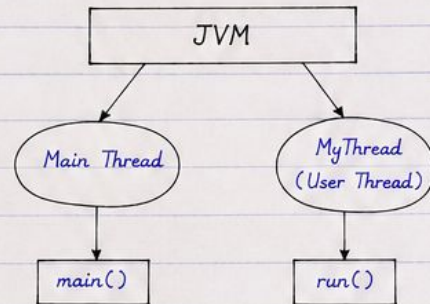
1. Thread class को extend करके:

- हम Thread class को extend करते हैं और run() method को override करते हैं।
- start() method call करने पर JVM एक नया thread create करता है और run() method को execute करता है।

Example:

```
class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Thread is running : " + i);
        }
    }
}

class TestThread1 {
    public static void main(String[] args) {
        MyThread t1 = new MyThread(); // Thread Object create
        t1.start(); // start() call
        System.out.println("Main thread is running");
    }
}
```



Note: start() method new thread को start करता है, run() method उसी thread के अंदर execute होता है।

2. Runnable interface को implement करके:

- Runnable interface को implement करते हैं और run() method को override करते हैं।
- Thread class का object बनाकर उसमें Runnable object pass करते हैं।

Example:

```
class MyRunnable implements Runnable {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println("Runnable thread running : " + i);
        }
    }
}

class TestThread2 {
    public static void main(String[] args) {
        MyRunnable r = new MyRunnable(); // Runnable Object
        Thread t = new Thread(r); // Thread Object
        t.start(); // start() call
        System.out.println("Main thread is running");
    }
}
```

Thread के Important Methods:

Method	Description
start()	Thread को start करने के लिए use होता है।
run()	Thread का कार्य (code) इसमें लिखा जाता है।
sleep(ms)	Thread को कुछ समय के लिए pause करता है।
join()	एक thread को दूसरे thread के पूरा होने तक wait करने के लिए use होता है।
isAlive()	Thread alive है या नहीं, boolean value देता है।
setPriority(p)	Thread की priority set करने के लिए।

Key Points:

- ✓ Thread object create करने के बाद start() method call करना जरूरी है।
- ✓ start() method एक बार ही call किया जा सकता है, दोबारा call करने पर IllegalThreadStateException आएगा।
- ✓ run() method को directly call करने पर नया thread नहीं बनता, वह सिर्फ method को normal तरीके से execute करता है।
- ✓ start() method call करने पर thread new path में run करता है, इसलिए multithreading achieve होती है।
- ✓ Thread बनाने के दो तरीके ही होते हैं: extend Thread class या implement Runnable interface।

1. Multithreading (मल्टीथ्रैडिंग) :

(A) Thread Priorities (थ्रेड प्राथमिकताएं) :

- हर thread की एक priority होती है जो बताती है कि वह thread कितनी महत्वपूर्ण है।
- Priority के आधार पर thread को CPU time मिलने की संभावना अधिक या कम होती है।
- Java में priority का range 1 (MIN_PRIORITY) से 10 (MAX_PRIORITY) तक होता है।
- Default priority NORM_PRIORITY (5) होती है।

Thread Priority Constants :

Constant	Value	Description
MIN_PRIORITY	1	सबसे कम प्राथमिकता
NORM_PRIORITY	5	सामान्य (Default) प्राथमिकता
MAX_PRIORITY	10	सबसे अधिक प्राथमिकता

Priority Scheduling :

- JVM scheduler high priority thread को low priority thread की तुलना में अधिक CPU time देता है।
- लेकिन यह guarantee नहीं है क्योंकि JVM implementation पर depend करता है।

Example :

```
class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(Thread.currentThread().getName() + " " + i);
        }
    }
}

class TestPriority {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();
        t1.setName("LowPriority");
        t2.setName("HighPriority");
        t1.setPriority(Thread.MIN_PRIORITY); // 1
        t2.setPriority(Thread.MAX_PRIORITY); // 10
        t1.start();
        t2.start();
    }
}
```

Priority Effect (संभावित परिणाम)



Note : Output order JVM और OS पर depend करता है।

More Example (Output will be different on different systems) :

Example 1: (High Priority may run first)

```
class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(Thread.currentThread().getName() + " running");
        }
    }
}

class PriorityDemo {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();
        t1.setName("A - Low");
        t2.setName("B - High");
        t1.setPriority(Thread.MIN_PRIORITY);
        t2.setPriority(Thread.MAX_PRIORITY);
        t1.start();
        t2.start();
    }
}
```

Example 2: (Default Priority)

```
class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(Thread.currentThread().getName() + " running");
        }
    }
}

class PriorityDemo2 {
    public static void main(String[] args) {
        MyThread t1 = new MyThread();
        MyThread t2 = new MyThread();
        t1.setName("T1");
        t2.setName("T2");
        // default priority NORM_PRIORITY (5)
        t1.start();
        t2.start();
    }
}
```

Points to Remember :

- ✓ setPriority(int newPriority) method से thread की priority set करते हैं।
- ✓ getPriority() method से thread की current priority प्राप्त करते हैं।
- ✓ Priority केवल suggestion होती है, guarantee नहीं होती कि high priority thread ही पहले execute होगा।
- ✓ Threads समान priority होने पर time slicing के आधार पर CPU time प्राप्त करते हैं।

Important Methods Related to Priority :

Method	Return Type	Description
setPriority(int newPriority)	void	Thread की priority set करने के लिए।
getPriority()	int	Thread की current priority प्राप्त करने के लिए।
getName()	String	Thread का नाम प्राप्त करने के लिए।
setName(String name)	void	Thread का नाम set करने के लिए।

1. Multithreading (मल्टीथ्रेडिंग) :

(B) Synchronizing Threads (थ्रेड का समन्वयन) :

- जब एक से अधिक threads किसी shared resource (data, object, file आदि) की access करती हैं, तो data inconsistency या race condition हो सकती हैं।
- Synchronization का उपयोग करके हम यह सुनिश्चित करते हैं कि एक समय पर केवल एक ही thread critical section (shared resource) को access करे।
- Synchronization के लिए Java में कुछ methods और keywords provide किए गए हैं।

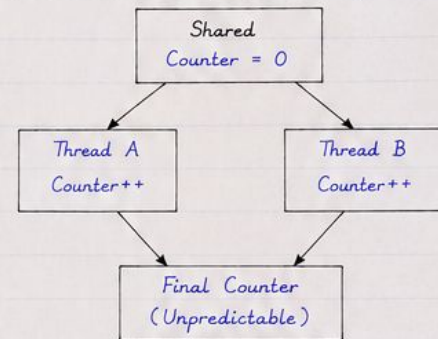
Why Synchronization is Needed ?

- Shared resource पर एक साथ access करने से गलत result मिल सकता है।
- Data corruption, inconsistent result और unpredictable behavior हो सकता है।

Ways to Synchronize Threads :

- synchronized keyword
- synchronized block
- synchronized method
- wait(), notify(), notifyAll() methods

Race Condition Example (Without Synchronization)



1. synchronized Keyword :

- synchronized keyword को method के साथ use करने पर पूरा method critical section बन जाता है।
- एक समय पर केवल एक ही thread उस method को execute कर सकती है।

Example :

```
class Counter {
    int count = 0;
    public synchronized void increment() {
        count++;
    }
}
// synchronized method
// यहाँ increment() method एक समय पर केवल एक
// thread द्वारा execute होगा।
```

2. synchronized Block :

- synchronized block केवल उस code भाग को critical section बनाता है जो block के अंदर है।
- इससे performance बेहतर रहती है क्योंकि पूरा method lock नहीं होता।

Example :

```
class Counter {
    int count = 0;
    public void increment() {
        synchronized(this) { // critical section
            count++;
        }
    }
}
// synchronized(this) का अर्थ है current object पर lock लगाना।
```

3. synchronized Method :

- synchronized method को दूसरे threads override नहीं कर सकते (lock behavior maintain करने के लिए)।

Example :

```
class Counter {
    public synchronized void show() {
        System.out.println("Counter Value");
    }
}
```

4. wait(), notify(), notifyAll() Methods :

- wait() : current thread को wait state में डाल देता है और lock release कर देता है।
- notify() : waiting threads में से एक को wake up करता है।
- notifyAll() : सभी waiting threads को wake up करता है।

Example using wait() and notify() :

```
class Example {
    public synchronized void display() throws InterruptedException {
        System.out.println("Thread waiting...");
        wait(); // thread wait में चला गया
        System.out.println("Thread notified...");
    }
    public synchronized void wakeup() {
        notify(); // waiting thread को जगाता है
    }
}
```

Thread States and Synchronization

State	Description
New	Thread create हुआ है।
Runnable	Thread run के लिए ready है।
Blocked / Waiting	Thread lock के लिए wait कर रहा है।
Running	Thread CPU पर execute हो रहा है।
Dead	Thread execution पूरा हो गया है।

Key Points :

- ✓ Synchronization race condition को रोकता है और data को सुरक्षित रखता है।
- ✓ synchronized keyword, synchronized block और synchronized method का उपयोग shared resource की safety के लिए किया जाता है।
- ✓ wait(), notify(), notifyAll() methods inter-thread communication के लिए उपयोगी होते हैं।